

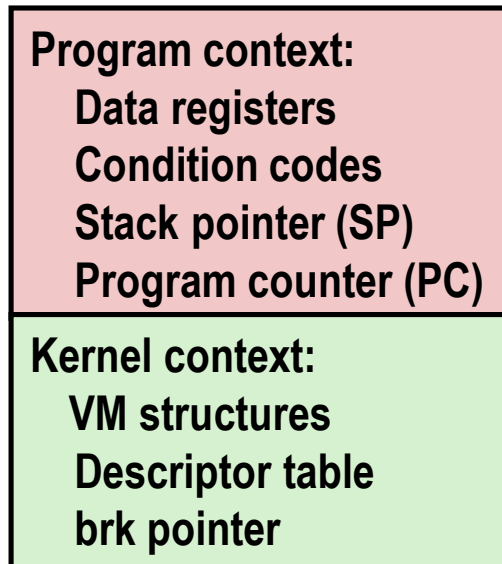
Synchronization: Basics

Introduction to Computer Systems
10th Lecture, Nov. 17, 2025

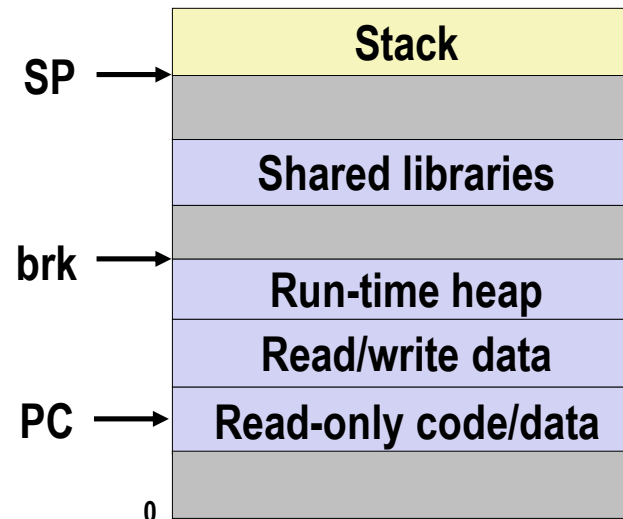
Traditional View of a Process

- Process = process context + code, data, and stack

Process context



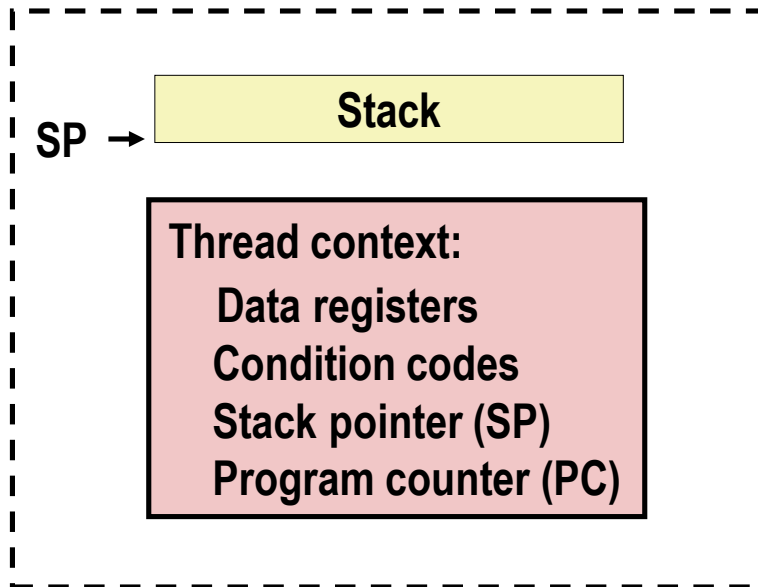
Code, data, and stack



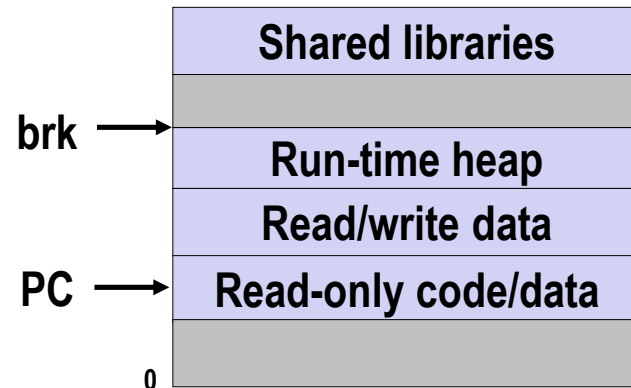
Alternate View of a Process

- Process = thread + code, data, and kernel context

Thread (main thread)



Code, data, and kernel context



Kernel context:
VM structures
Descriptor table
brk pointer

A Process With Multiple Threads

- **Multiple threads can be associated with a process**
 - Each thread has its own logical control flow
 - Each thread shares the same code, data, and kernel context
 - Each thread has its own stack for local variables
 - but not protected from other threads
 - Each thread has its own thread id (TID)

Thread 1 (main thread)

Thread 2 (peer thread)

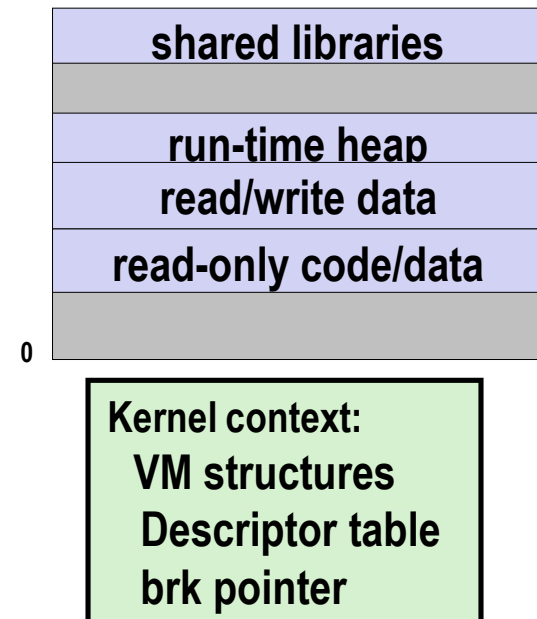
stack 1

stack 2

Thread 1 context:
Data registers
Condition codes
SP1
PC1

Thread 2 context:
Data registers
Condition codes
SP2
PC2

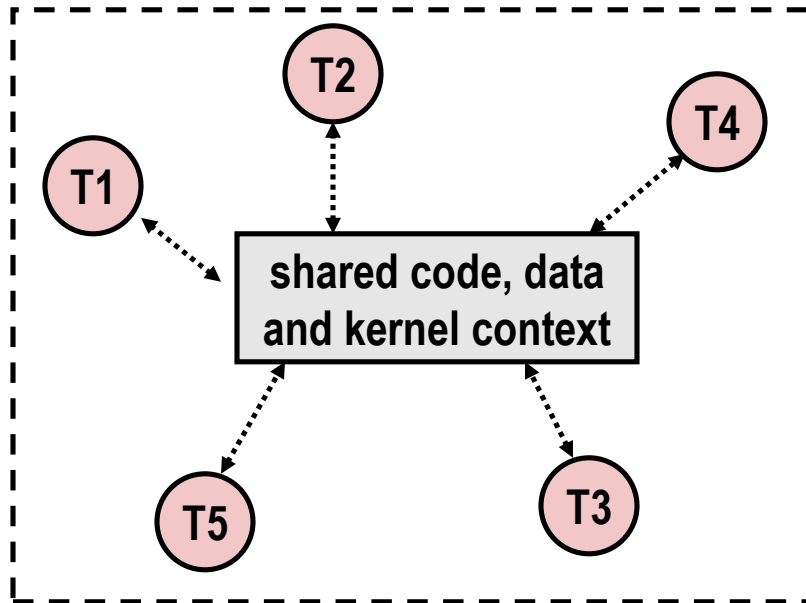
Shared code and data



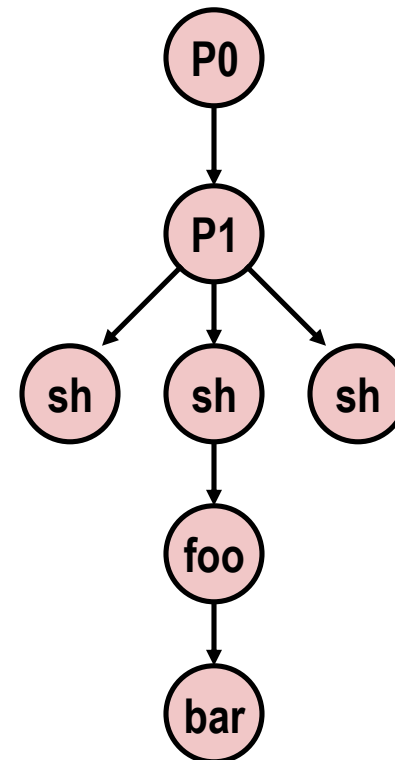
Logical View of Threads

- **Threads associated with process form a pool of peers**
 - Unlike processes which form a tree hierarchy

Threads associated with process foo



Process hierarchy



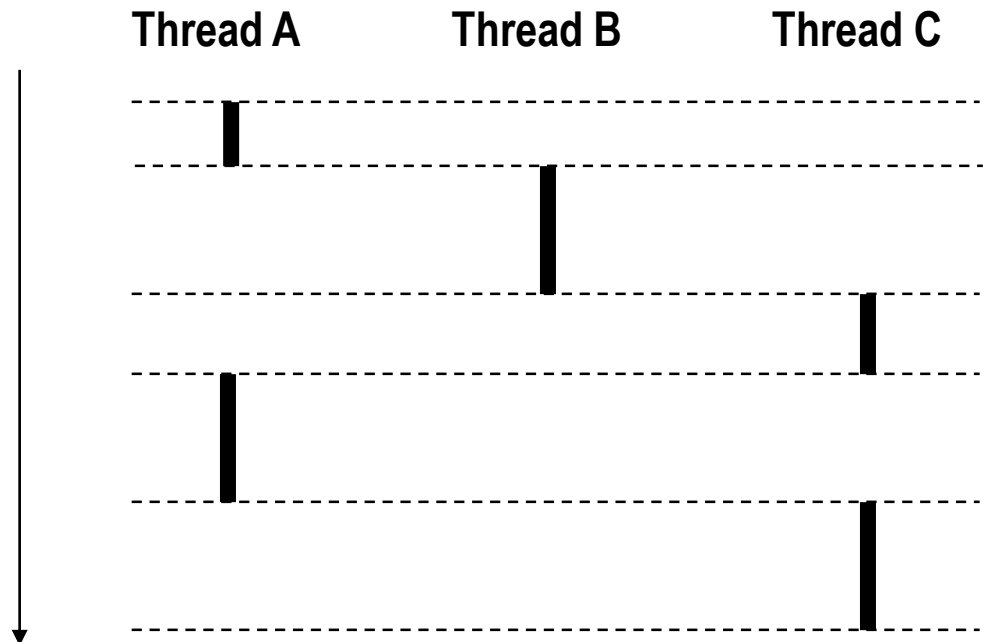
Concurrent Threads

- Two threads are concurrent if their flows overlap in time
- Otherwise, they are sequential

- **Examples:**

- Concurrent: A & B, A&C
- Sequential: B & C

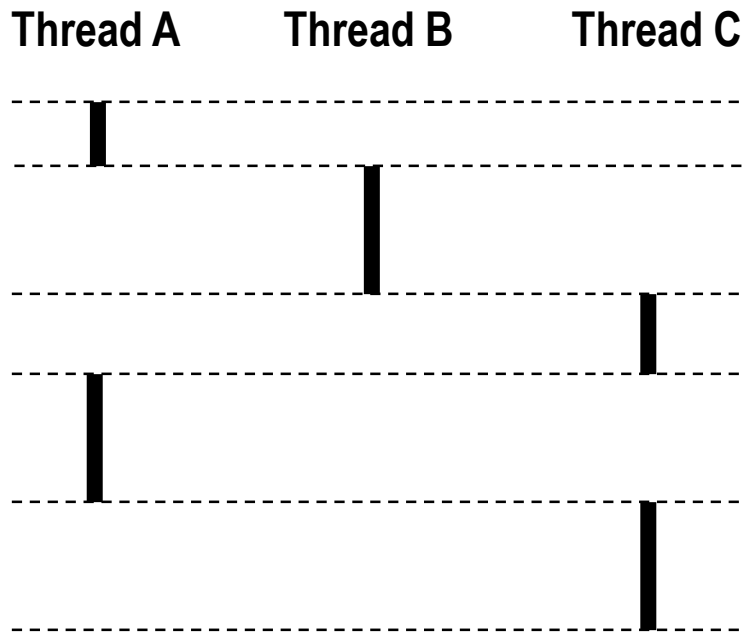
Time



Concurrent Thread Execution

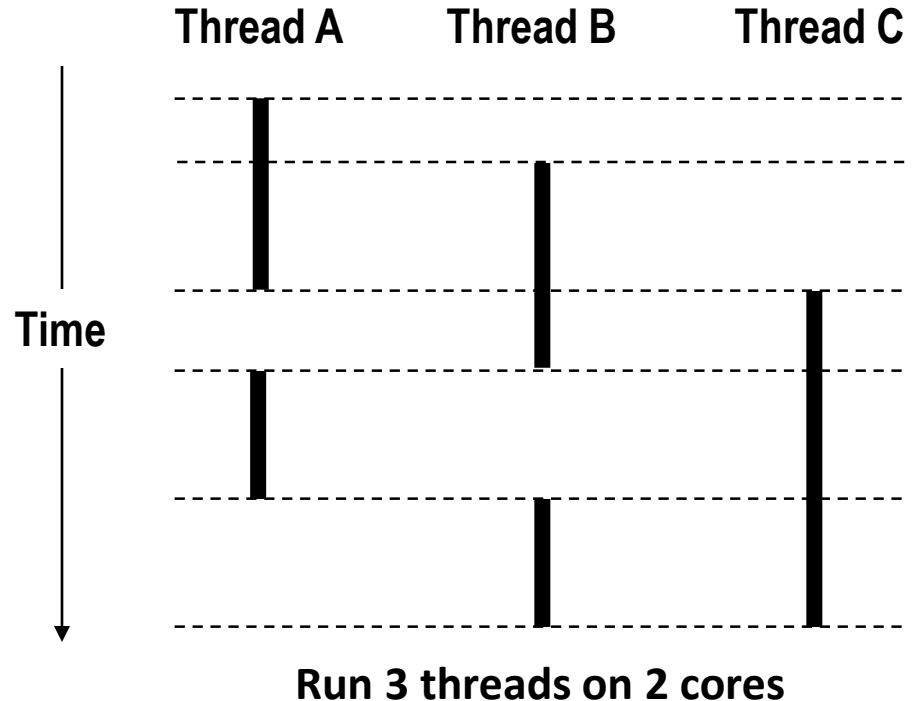
■ Single Core Processor

- Simulate parallelism by time slicing



■ Multi-Core Processor

- Can have true parallelism



Threads vs. Processes

■ How threads and processes are similar

- Each has its own logical control flow
- Each can run concurrently with others (possibly on different cores)
- Each is context switched

■ How threads and processes are different

- Threads share all code and data (except local stacks)
 - Processes (typically) do not
- Threads are somewhat less expensive than processes
 - Process control (creating and reaping) twice as expensive as thread control
 - Linux numbers:
 - ~20K cycles to create and reap a process
 - ~10K cycles (or less) to create and reap a thread

Posix Threads (Pthreads) Interface

- ***Pthreads: Standard interface for ~60 functions that manipulate threads from C programs***
 - Creating and reaping threads
 - *pthread_create()*
 - *pthread_join()*
 - Determining your thread ID
 - *pthread_self()*
 - Terminating threads
 - *pthread_cancel()*
 - *pthread_exit()*
 - *exit()* [terminates all threads] , *RET* [terminates current thread]
 - Synchronizing access to shared variables
 - *pthread_mutex_init*
 - *pthread_mutex_[un]lock*

The Pthreads "hello, world" Program

```
/*  
 * hello.c - Pthreads "hello, world" program  
 */
```

```
#include "csapp.h"
```

```
void *thread(void *vargp);
```

```
int main()  
{
```

```
    pthread_t tid;
```

```
    Pthread_create(&tid, NULL, thread, NULL);
```

```
    Pthread_join(tid, NULL);
```

```
    exit(0);
```

```
}
```

hello.c

Thread ID

Thread attributes
(usually NULL)

Thread routine

Thread arguments
(void *p)

Return value
(void **p)

```
void *thread(void *vargp) /* thread routine */
```

```
{
```

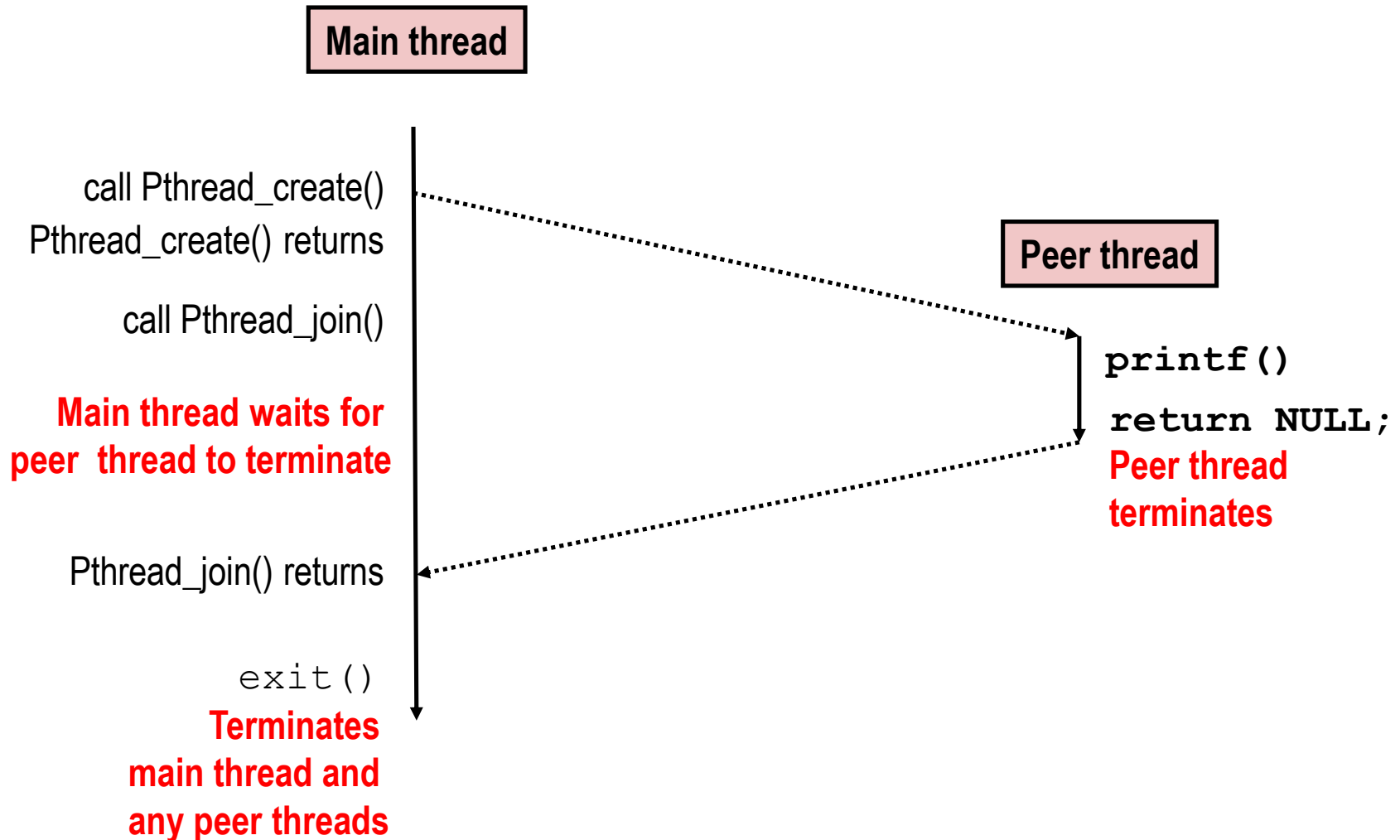
```
    printf("Hello, world!\n");
```

```
    return NULL;
```

```
}
```

hello.c

Execution of Threaded “hello, world”



Shared Variables in Threaded C Programs

- **Question: Which variables in a threaded C program are shared?**
 - The answer is not as simple as “*global variables are shared*” and “*stack variables are private*”
- **Def: A variable x is *shared* if and only if multiple threads reference some instance of x .**
- **Requires answers to the following questions:**
 - What is the memory model for threads?
 - How are instances of variables mapped to memory?
 - How many threads might reference each of these instances?

Threads Memory Model

■ Conceptual model:

- Multiple threads run within the context of a single process
- Each thread has its own separate thread context
 - Thread ID, stack, stack pointer, PC, condition codes, and GP registers
- All threads share the remaining process context
 - Code, data, heap, and shared library segments of the process virtual address space
 - Open files and installed handlers

■ Operationally, this model is not strictly enforced:

- Register values are truly separate and protected, but...
- Any thread can read and write the stack of any other thread

The mismatch between the conceptual and operation model is a source of confusion and errors

Example Program to Illustrate Sharing

```
char **ptr; /* global var */
```

```
int main()
```

```
{
```

```
    long i;
```

```
    pthread_t tid;
```

```
    char *msgs[2] = {  
        "Hello from foo",  
        "Hello from bar"
```

```
};
```

```
    ptr = msgs;
```

```
    for (i = 0; i < 2; i++)
```

```
        Pthread_create(&tid,
```

```
            NULL,
```

```
            thread,
```

```
            (void *)i);
```

```
    Pthread_exit(NULL);
```

```
}
```

sharing.c

```
void *thread(void *vargp)
```

```
{
```

```
    long myid = (long)vargp;
```

```
    static int cnt = 0;
```

```
    printf("[%ld]: %s (cnt=%d)\n",
```

```
        myid, ptr[myid], ++cnt);
```

```
    return NULL;
```

```
}
```



Peer threads reference main thread's stack indirectly through global ptr variable

Mapping Variable Instances to Memory

■ Global variables

- *Def:* Variable declared outside of a function
- **Virtual memory contains exactly one instance of any global variable**

■ Local variables

- *Def:* Variable declared inside function without `static` attribute
- **Each thread stack contains one instance of each local variable**

■ Local static variables

- *Def:* Variable declared inside function with the `static` attribute
- **Virtual memory contains exactly one instance of any local static variable.**

Mapping Variable Instances to Memory

Global var: 1 instance (`ptr` [data])

```
char **ptr; /* global var */
```

```
int main()
```

```
{  
    long i;  
    pthread_t tid;  
    char *msgs[2] = {  
        "Hello from foo",  
        "Hello from bar"  
    };  
  
    ptr = msgs;  
    for (i = 0; i < 2; i++)  
        Pthread_create(&tid,  
            NULL,  
            thread,  
            (void *)i);  
    Pthread_exit(NULL);  
}
```

sharing.c

Local vars: 1 instance (`i.m`, `msgs.m`)

Local var: 2 instances (
 `myid.p0` [peer thread 0's stack],
 `myid.p1` [peer thread 1's stack]
)

```
void *thread(void *vargp)  
{  
    long myid = (long)vargp;  
    static int cnt = 0;  
  
    printf("[%ld]: %s (cnt=%d)\n",  
        myid, ptr[myid], ++cnt);  
    return NULL;  
}
```

Local static var: 1 instance (`cnt` [data])

Shared Variable Analysis

■ Which variables are shared?

<i>Variable instance</i>	<i>Referenced by main thread?</i>	<i>Referenced by peer thread 0?</i>	<i>Referenced by peer thread 1?</i>
<code>ptr</code>	yes	yes	yes
<code>cnt</code>	no	yes	yes
<code>i.m</code>	yes	no	no
<code>msgs.m</code>	yes	yes	yes
<code>myid.p0</code>	no	yes	no
<code>myid.p1</code>	no	no	yes

■ Answer: A variable **x** is shared if multiple threads reference at least one instance of **x**. Thus:

- `ptr`, `cnt`, and `msgs` are shared
- `i` and `myid` are *not* shared

Synchronizing Threads

- Shared variables are handy...
- ...but introduce the possibility of nasty *synchronization* errors.

badcnt.c: Improper Synchronization

```
/* Global shared variable */
volatile long cnt = 0; /* Counter */

int main(int argc, char **argv)
{
    long niters;
    pthread_t tid1, tid2;

    niters = atoi(argv[1]);
    Pthread_create(&tid1, NULL,
        thread, &niters);
    Pthread_create(&tid2, NULL,
        thread, &niters);
    Pthread_join(tid1, NULL);
    Pthread_join(tid2, NULL);

    /* Check result */
    if (cnt != (2 * niters))
        printf("BOOM! cnt=%ld\n", cnt);
    else
        printf("OK cnt=%ld\n", cnt);
    exit(0);
}
```

badcnt.c

```
/* Thread routine */
void *thread(void *vargp)
{
    long i, niters =
        *((long *)vargp);

    for (i = 0; i < niters; i++)
        cnt++;

    return NULL;
}
```

```
linux> ./badcnt 10000
OK cnt=20000
linux> ./badcnt 10000
BOOM! cnt=13051
linux>
```

cnt should equal 20,000.
What went wrong?

Assembly Code for Counter Loop

C code for counter loop in thread i

```
for (i = 0; i < niters; i++)  
    cnt++;
```

Asm code for thread i

<pre>movq (%rdi), %rcx testq %rcx,%rcx jle .L2 movl \$0, %eax</pre>	} H_i : Head	
<pre>----- .L3: movq cnt(%rip), %rdx addq \$1, %rdx movq %rdx, cnt(%rip)</pre>		} L_i : Load cnt U_i : Update cnt S_i : Store cnt
<pre>----- addq \$1, %rax cmpq %rcx, %rax jne .L3</pre>		
<pre>.L2:</pre>		

Concurrent Execution

- **Key idea:** In general, any sequentially consistent interleaving is possible, but some give an unexpected result!
 - I_i denotes that thread i executes instruction I
 - $\%rdx_i$ is the content of $\%rdx$ in thread i 's context

i (thread)	$instr_i$	$\%rdx_1$	$\%rdx_2$	cnt
1	H_1	-	-	0
1	L_1	0	-	0
1	U_1	1	-	0
1	S_1	1	-	1
2	H_2	-	-	1
2	L_2	-	1	1
2	U_2	-	2	1
2	S_2	-	2	2
2	T_2	-	2	2
1	T_1	1	-	2



Thread 1
critical section



Thread 2
critical section

OK

Concurrent Execution (cont)

- Incorrect ordering: two threads increment the counter, but the result is 1 instead of 2

i (thread)	instr _i	%rdx ₁	%rdx ₂	cnt
1	H ₁	-	-	0
1	L ₁	0	-	0
1	U ₁	1	-	0
2	H ₂	-	-	0
2	L ₂	-	0	0
1	S ₁	1	-	1
1	T ₁	1	-	1
2	U ₂	-	1	1
2	S ₂	-	1	1
2	T ₂	-	1	1

Oops!

Concurrent Execution (cont)

- How about this ordering?

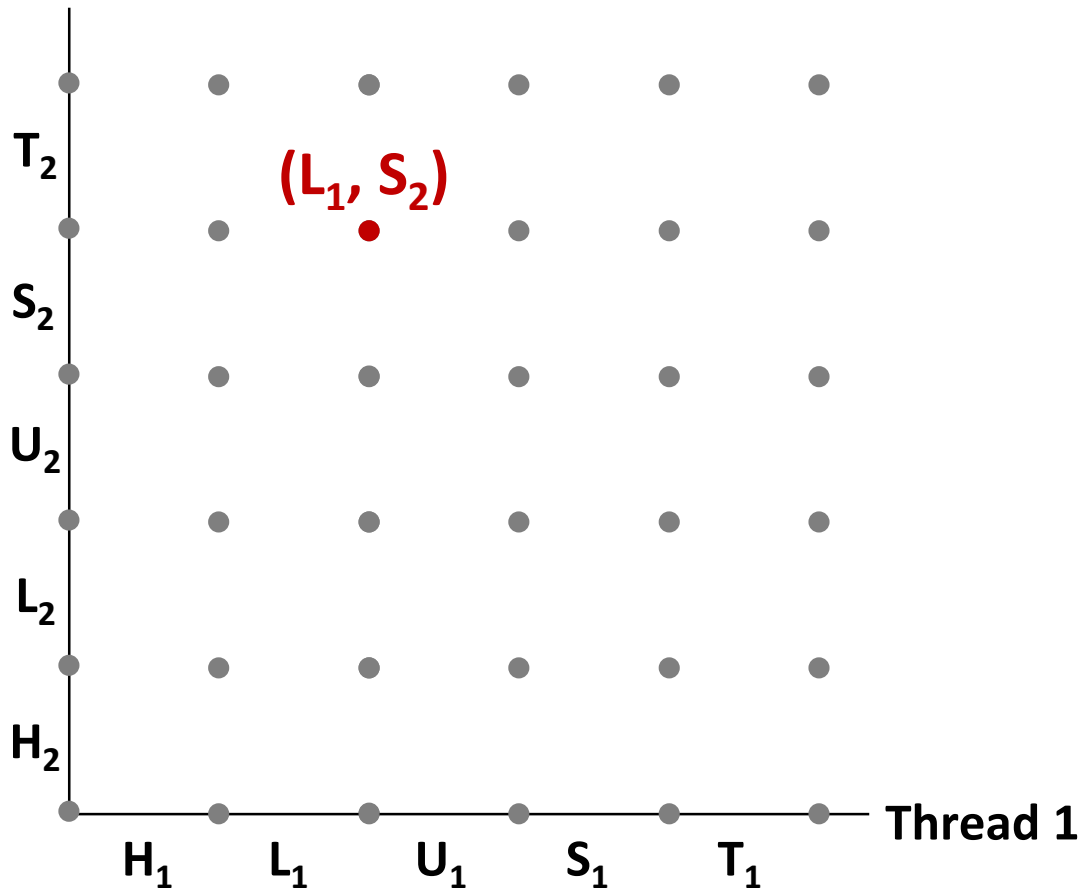
i (thread)	instr _i	%rdx ₁	%rdx ₂	cnt
1	H ₁			0
1	L ₁	0		
2	H ₂			
2	L ₂		0	
2	U ₂		1	
2	S ₂		1	1
1	U ₁	1		
1	S ₁	1		1
1	T ₁			1
2	T ₂			1

Oops!

- We can analyze the behavior using a *progress graph*

Progress Graphs

Thread 2



A *progress graph* depicts the discrete *execution state space* of concurrent threads.

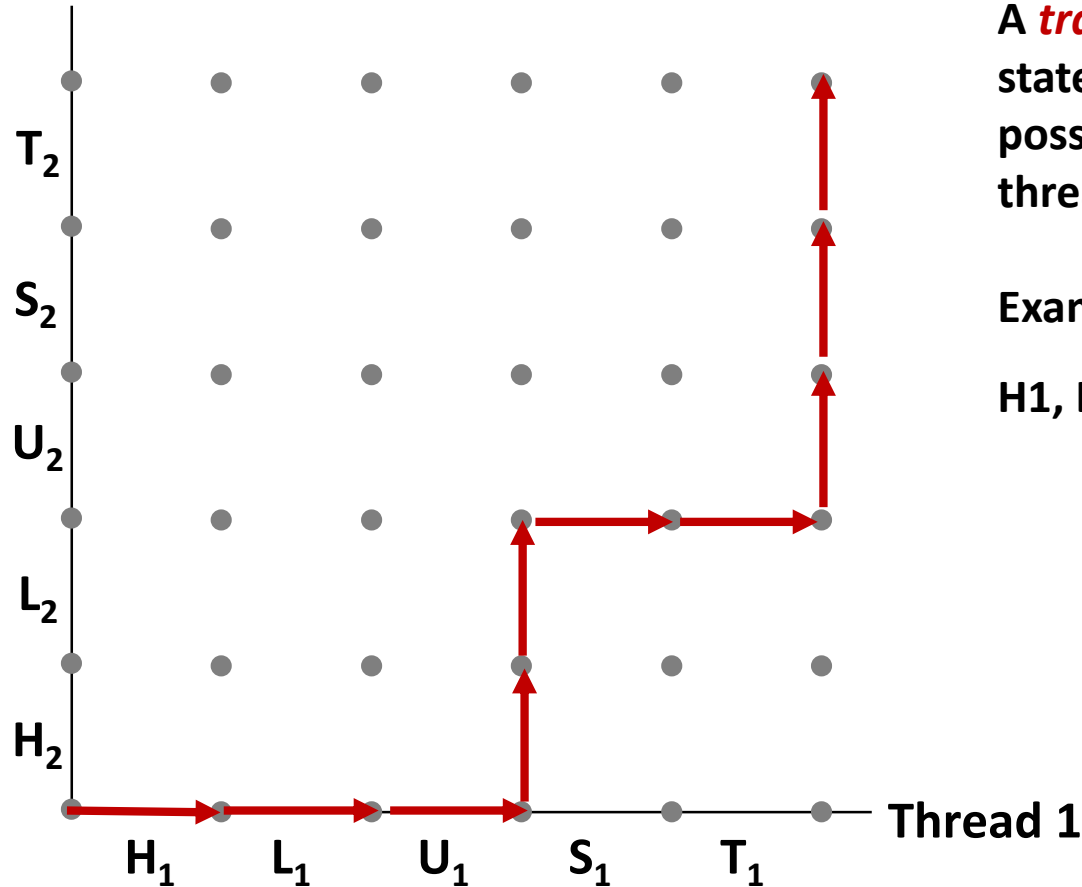
Each axis corresponds to the sequential order of instructions in a thread.

Each point corresponds to a possible *execution state* ($\text{Inst}_1, \text{Inst}_2$).

E.g., (L_1, S_2) denotes state where thread 1 has completed L_1 and thread 2 has completed S_2 .

Trajectories in Progress Graphs

Thread 2

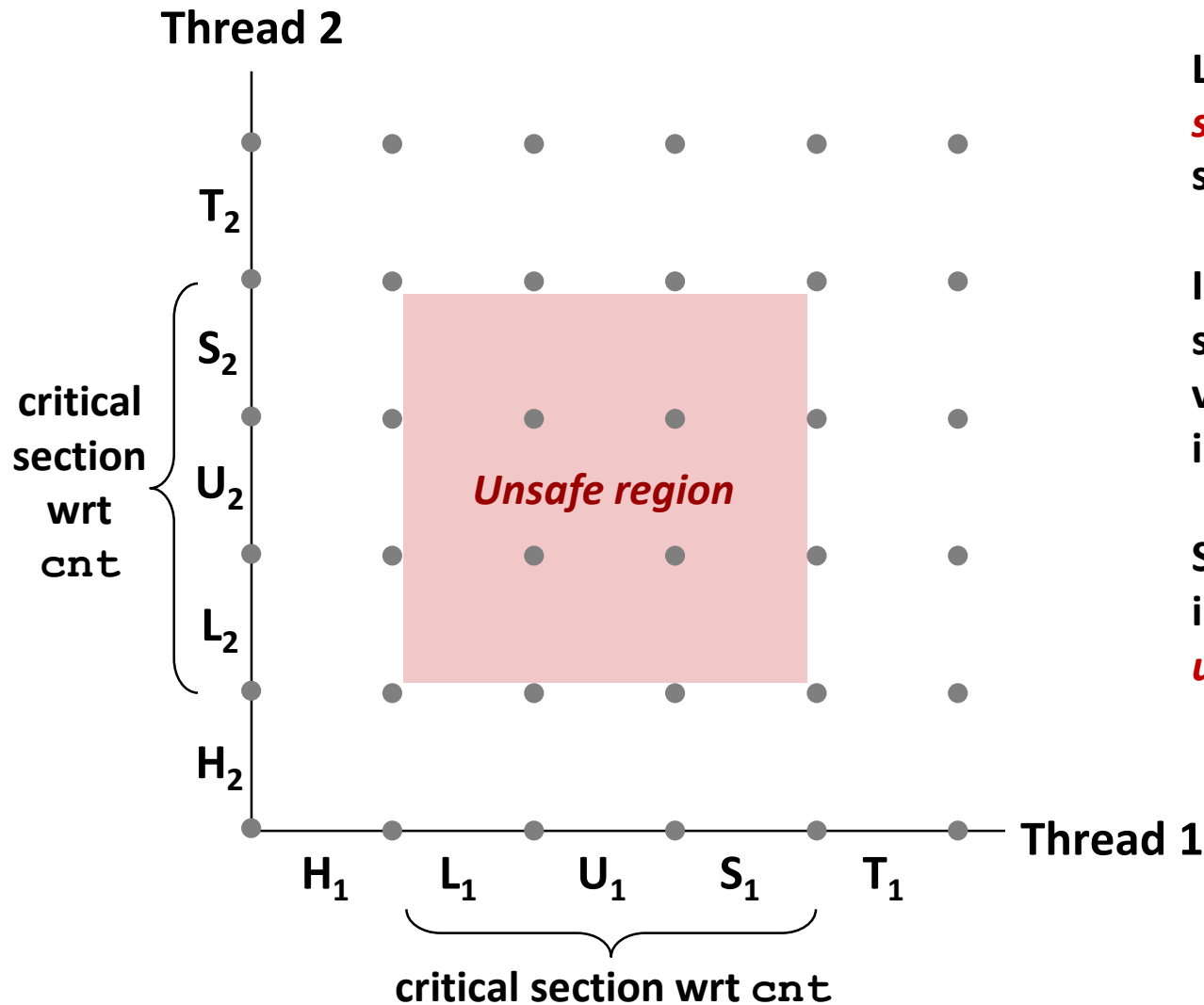


A *trajectory* is a sequence of legal state transitions that describes one possible concurrent execution of the threads.

Example:

$H_1, L_1, U_1, H_2, L_2, S_1, T_1, U_2, S_2, T_2$

Critical Sections and Unsafe Regions

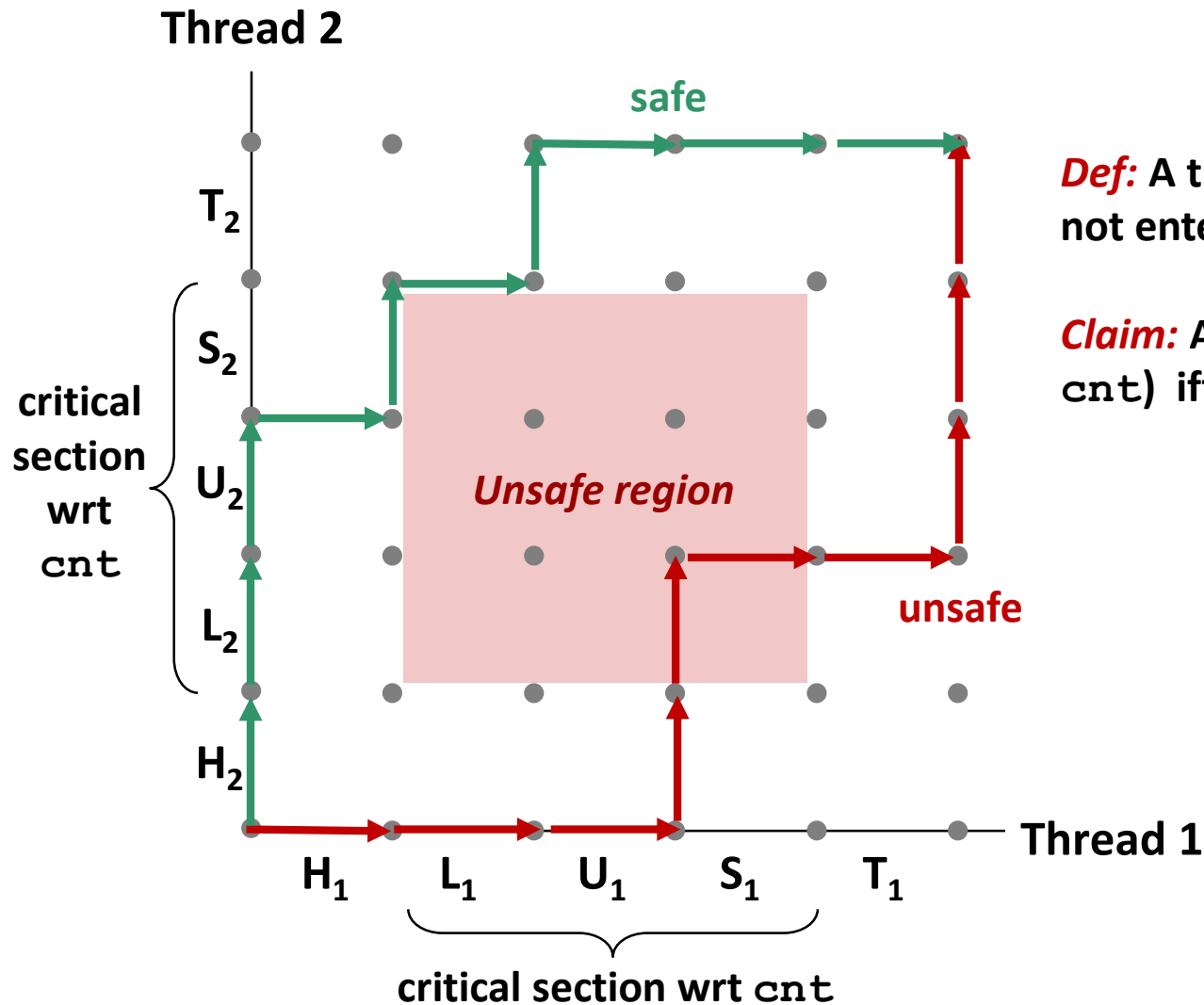


L, U, and S form a **critical section** with respect to the shared variable `cnt`

Instructions in critical sections (wrt some shared variable) should not be interleaved

Sets of states where such interleaving occurs form **unsafe regions**

Critical Sections and Unsafe Regions



Def: A trajectory is *safe* iff it does not enter any unsafe region

Claim: A trajectory is correct (wrt cnt) iff it is safe

Enforcing Mutual Exclusion

- **Question:** How can we guarantee a safe trajectory?
- **Answer:** We must *synchronize* the execution of the threads so that they can never have an unsafe trajectory.
 - i.e., need to guarantee *mutually exclusive access* for each critical section.
- **Classic solution:**
 - Semaphores (Edsger Dijkstra)
- **Other approaches (out of our scope)**
 - Mutex and condition variables (Pthreads)
 - Monitors (Java)

Semaphores

- ***Semaphore***: non-negative global integer synchronization variable. Manipulated by *P* and *V* operations.
- ***P(s)***
 - If *s* is nonzero, then decrement *s* by 1 and return immediately.
 - Test and decrement operations occur atomically (indivisibly)
 - If *s* is zero, then suspend thread until *s* becomes nonzero and the thread is restarted by a *V* operation.
 - After restarting, the *P* operation decrements *s* and returns control to the caller.
- ***V(s)***:
 - Increment *s* by 1.
 - Increment operation occurs atomically
 - If there are any threads blocked in a *P* operation waiting for *s* to become non-zero, then restart exactly one of those threads, which then completes its *P* operation by decrementing *s*.
- **Semaphore invariant: ($s \geq 0$)**

C Semaphore Operations

Pthreads functions:

```
#include <semaphore.h>

int sem_init(sem_t *s, 0, unsigned int val);} /* s = val */

int sem_wait(sem_t *s); /* P(s) */
int sem_post(sem_t *s); /* V(s) */
```

badcnt.c: Improper Synchronization

```
/* Global shared variable */
volatile long cnt = 0; /* Counter */

int main(int argc, char **argv)
{
    long niters;
    pthread_t tid1, tid2;

    niters = atoi(argv[1]);
    Pthread_create(&tid1, NULL,
        thread, &niters);
    Pthread_create(&tid2, NULL,
        thread, &niters);
    Pthread_join(tid1, NULL);
    Pthread_join(tid2, NULL);

    /* Check result */
    if (cnt != (2 * niters))
        printf("BOOM! cnt=%ld\n", cnt);
    else
        printf("OK cnt=%ld\n", cnt);
    exit(0);
}
```

badcnt.c

```
/* Thread routine */
void *thread(void *vargp)
{
    long i, niters =
        *((long *)vargp);

    for (i = 0; i < niters; i++)
        cnt++;

    return NULL;
}
```

How can we fix this using semaphores?

Using Semaphores for Mutual Exclusion

■ Basic idea:

- Associate a unique semaphore *mutex*, initially 1, with each shared variable (or related set of shared variables).
- Surround corresponding critical sections with $P(mutex)$ and $V(mutex)$ operations.

■ Terminology:

- *Binary semaphore*: semaphore whose value is always 0 or 1
- *Mutex*: binary semaphore used for mutual exclusion
 - P operation: “locking” the mutex
 - V operation: “unlocking” or “releasing” the mutex
 - “Holding” a mutex: locked and not yet unlocked.
- *Counting semaphore*: used as a counter for set of available resources.

goodcnt.c: Proper Synchronization

- Define and initialize a mutex for the shared variable cnt:

```
volatile long cnt = 0; /* Counter */
sem_t mutex;          /* Semaphore that protects cnt */

Sem_init(&mutex, 0, 1); /* mutex = 1 */
```

- Surround critical section with *P* and *V*:

```
for (i = 0; i < niters; i++) {
    P(&mutex);
    cnt++;
    V(&mutex);
}
```

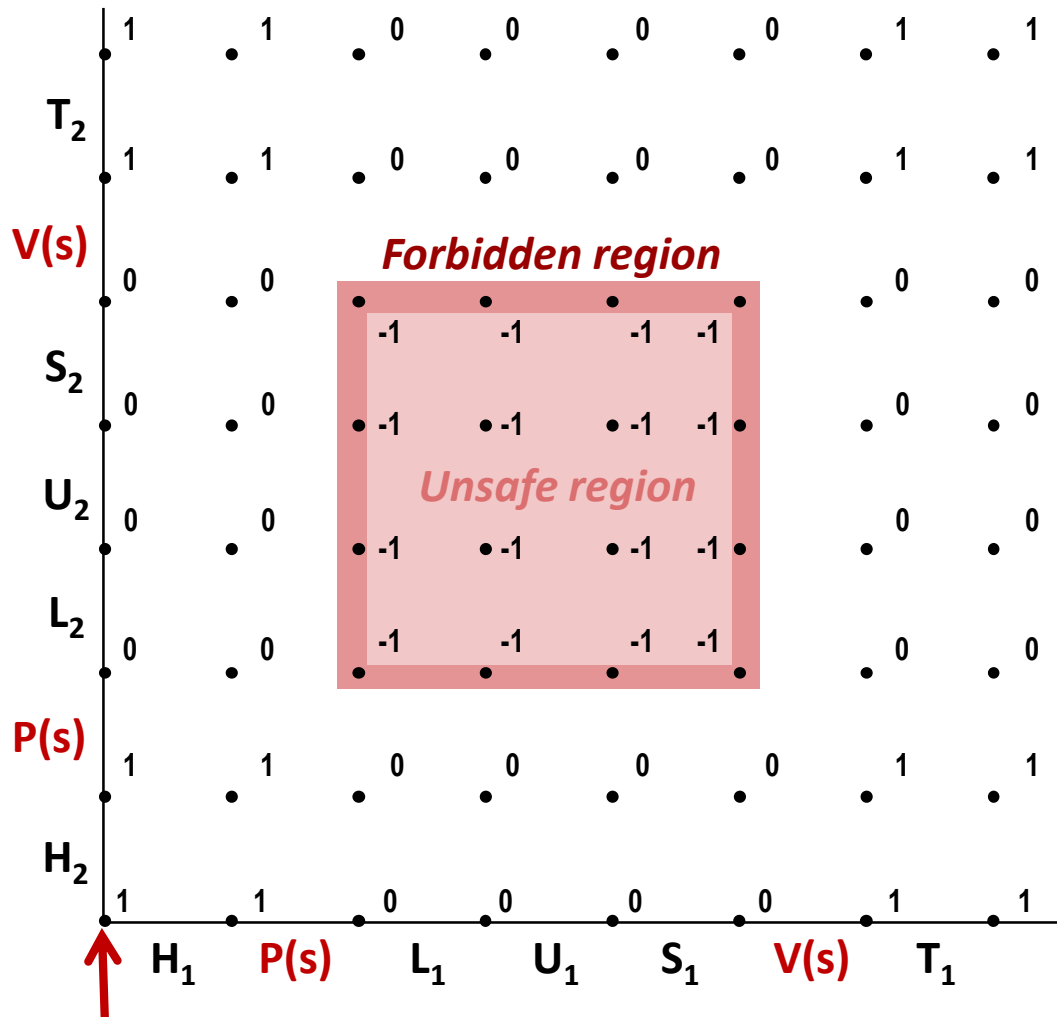
goodcnt.c

```
linux> ./goodcnt 10000
OK cnt=20000
linux> ./goodcnt 10000
OK cnt=20000
linux>
```

Warning: It's orders of magnitude slower than badcnt.c.

Why Mutexes Work

Thread 2



Initially $s = 1$

Provide mutually exclusive access to shared variable by surrounding critical section with P and V operations on semaphore s (initially set to 1)

Semaphore invariant creates a **forbidden region** that encloses unsafe region and that cannot be entered by any trajectory.

Thread 1

Summary

- **Programmers need a clear model of how variables are shared by threads.**
- **Variables shared by multiple threads must be protected to ensure mutually exclusive access.**
- **Semaphores are a fundamental mechanism for enforcing mutual exclusion.**